

Netra: Network Traffic Monitoring Platform Report

Scope of Project

Netra is a real-time network traffic monitoring and analysis platform. It provides a user-friendly dashboard to capture, filter, and visualize live network packets. It allows for the analysis of network protocols, active ports, top communication sources, and overall data transfer metrics, bridging the gap between raw packet sniffing and accessible network analytics.

Dataset Description

The dataset is dynamically captured from live network interfaces and stored persistently in "network_traffic.csv". This CSV file serves as the historical database containing records of captured packets.

Fields:

- Timestamp: The time the packet was captured.
- Source IP & Destination IP: Origin and target addresses.
- Protocol: Transport layer protocol (TCP, UDP, ICMP).
- Packet Size: Total byte size of the packet.
- Source & Destination Port: Logical endpoints indicating the service type (e.g., HTTP, HTTPS).

Tech Stack & Details

- Backend Core: Python
- Web Framework: Flask, serving REST API endpoints.
- Packet Sniffing: Scapy, utilized in a background daemon thread for non-blocking capture.
- Data Processing: Pandas, used for efficient data aggregation and statistical analysis.
- Frontend UI: HTML, Vanilla CSS (with modern Glassmorphism aesthetics), and JavaScript (Fetch API for dynamic, asynchronous updates).
- Storage: CSV for raw network packet logs and JSON for persistent application usage statistics.

Methodology

1. Foundation Setup: Defined the application architecture and initialized the Flask server.
2. Sniffer Engine: Built a background threaded packet sniffer using Scapy to capture live traffic, extract IP/Port/Protocol details, and maintain an in-memory ring buffer.
3. Data Storage Mechanism: Implemented automatic and thread-safe logging of packet metrics into "network_traffic.csv".
4. RESTful API Development: Created API endpoints in Flask to stream live packets, deliver historical data, and serve aggregated statistical analytics via Pandas.
5. Frontend Design: Developed a modern, dark-themed responsive UI with dynamic traffic tables, live filter controls, and visual progress bars for protocol and port statistics.

Netra: Network Traffic Monitoring Platform Report

Challenges Faced

- Concurrency & UI Freezes: Raw packet sniffing using Scapy is blocking and computationally intensive.
Solution: Moved the sniffing process to a background daemon thread with a dedicated in-memory ring buffer so it doesn't block Flask API requests.
- Browser Caching Issues: The frontend dashboard occasionally displayed stale data after restarting a capture session due to aggressive GET request caching.
Solution: Implemented dynamic timestamp cache-busting on all background Fetch requests to ensure data freshness.
- Data Overload: Live network traffic generates thousands of packets per minute, causing frontend UI lag when rendering DOM elements.
Solution: Limited the frontend live view to the 50 most recent packets while maintaining accurate total statistics in the backend memory.

Future Work

- Deep Packet Inspection (DPI): Implement functionality to inspect and identify application-layer payloads.
- Threat Detection: Add machine learning capabilities or signature matching for anomaly detection (e.g., identifying potential DDoS attacks or Port Scanning).
- Interactive Visualizations: Introduce interactive graphical charts for time-series visualization of network throughput and spikes.

References

- Scapy: Packet manipulation and analysis library.
- Flask: Micro web framework for the backend server.
- Pandas: High-performance data manipulation and analysis library.